

Detecting KillAura, Flight, and X-Ray Cheats in Minecraft Java Edition: Using Keystroke & Mouse Behavioral Biometrics With Machine Learning

Timothy Britcliffe

Undergraduate Student

Computing Science, Thompson Rivers University

Kpughur-Tule Tertseghe Moses

Undergraduate Student

Computing Science, Thompson Rivers University

April 16th, 2026

1 Abstract

The project focuses on Minecraft Java Edition and employs keystroke dynamics and mouse movement patterns to detect three common cheats: KillAura, Flight, and X-Ray. Real gameplay data was collected from participants using the Wurst Client to enable the cheats, with an external Python logger capturing raw mouse and keyboard events. The data underwent preprocessing to remove idle periods and apply feature scaling. Timing features were then extracted from the keyboard data, including mean dwell time, dwell time variance, dwell uniformity, mean flight time, flight time variance, flight uniformity, and event density per window. Geometric and movement features were extracted from the mouse data, including path length, straightness ratio, mean speed, maximum jerk, mean and standard deviation of angle changes, and heading stability. Separate LightGBM classifiers were trained for the keyboard and mouse feature sets. These models were evaluated using cross-validation, yielding strong performance on keyboard features (around 83–85 %) but lower and more variable performance on mouse features (50–68 %). At test time, the probability outputs from both models were averaged for each 5-second window. A window was considered suspicious if the maximum averaged probability met or exceeded a 0.5 threshold; if more than 50 % of the windows in a full session were flagged as suspicious, the session was labeled as cheating. The system successfully flagged KillAura and Flight sessions in most cases but struggled with X-Ray due to its similarity to both legitimate baseline gameplay and Flight patterns. The

final outputs show that behavioral biometrics can be used to effectively detect certain cheats in Minecraft, while also displaying the current limitations that the system might have.

2 Introduction

Minecraft: Java Edition is one of the largest and most widely played video games and has been for more than a decade. More specifically, its online/multiplayer modes (via Minecraft Realms and private servers) have a massive user base. These large player counts make the gameplay significantly more susceptible to a variety of cheats, which give players unfair advantages over those who do not want to cheat. This paper focuses on three of the most common methods of cheating in Minecraft: Java Edition [1]. The first method is KillAura, which allows the user to perform automated attacks on nearby entities, generally within a limited range. The second method is flight, a type of movement hack that enables the user to fly in game when they should not be able to. The third method is X-Ray, which allows users to see through opaque blocks, enabling them to locate nearby structures and high-rarity items that would otherwise be hidden. Behavioral biometrics offer a reliable client-side method of detecting these cheats, as all three methods cause irregularities in both keystroke and mouse movement patterns. Prior work on using mouse movements for continuous authentication in Minecraft achieved an accuracy rate of 92% using Random Forest on real gameplay data from 10 players [2]. A related implementation already exists that uses keystrokes and mouse movement to identify whether the user playing a game is the owner of the account, specifically for the game League of Legends [3]. However, while that work was focused on user authentication (for verifying the account owner), this project targets cheat detection by identifying abnormal behavior that can indicate cheating. This project aims to build on prior research by incorporating keystroke dynamics and extending the scope to detect the three specific methods of cheating described above. The goal is to classify normal user behavior against the behavior of a user employing cheats. The Wurst Client and the Impact Client (two notable hacked clients for Minecraft: Java Edition) will be used to test the system’s capabilities, with a target average accuracy rate of 90%+ across all three cheating methods.

3 Literature Review

There is no prior work that directly focuses on detecting KillAura, Flight, and X-Ray cheats in Minecraft: Java Edition using mouse and keyboard behavioral biometrics. However, closely related research has shown that in-game input, particularly mouse input, can function as a method of behavioral biometrics and can be used in tasks such as player identification, authentication, and cheat detection [4]. Siddiqui et al. introduced this functionality and applied it to continuous authentication, training a Random Forest classifier for each user to

distinguish genuine game interactions from those of another player or spoofed interactions. In that work, Siddiqui et al. collected data from 10 participants who each played Minecraft on a fixed desktop setup for 20 minutes while a Python based external logger recorded mouse movement, clicks, and scroll events with timestamps and on-screen coordinates [2]. Each user had a Random Forest classifier trained to distinguish their actions from a set of imposter actions drawn from the remaining participants [2]. This authentication method achieved 92% accuracy and produced fairly low equal error rates and lower false positive rates compared to several prior mouse dynamic datasets [2]. This result supported the concept that mouse behavior during gameplay can capture user-specific patterns that a machine learning model can reliably identify. The Siddiqui study has several limitations with respect to our research goal of cheat detection. First, their problem space is strictly “Which human is this?” and does not address hacked clients, automated behaviors, or macros. All input in their dataset is purely human, so their models are optimized to differentiate one player’s behavior from another’s rather than distinguishing human control from computer control [2]. Second, they use only one modality, mouse biometrics, and do not log keyboard timing or press patterns, even though movement hacks tend to manifest more clearly in keystroke rhythm [3]. Third, participants were allowed to perform any in-game activity during the session, combining combat, building, and menu navigation without specific labels for each. This approach works well for authentication but is less suited to isolating the specific irregularities introduced by KillAura, Flight, or X-Ray cheats, which are tied to specific tasks such as mining paths and PvP aiming. Other work has explored behavioral biometric methods in video games using modalities beyond mouse movement. Beserra et al. used both mouse biometrics and keystroke biometrics for user identification in the multiplayer game League of Legends, demonstrating that combining keyboard and mouse biometrics can yield favorable results compared to mouse biometrics alone [3]. This also supports the view that keyboard dynamics provide valuable behavioral patterns for user identification. Our project extends this prior work in three primary ways. First, instead of framing the task as identity verification or authentication [2, 3], we treat behavioral biometrics as a channel for cheat detection: the core question is whether the input stream looks human or exhibits the computer-generated irregularities introduced by hacked clients such as the Wurst or Impact clients. Second, we adopt a multimodal approach that combines the mouse based analysis of Siddiqui et al. with the joint mouse and keyboard approach demonstrated by Beserra et al., logging both cursor trajectories and keystroke dynamics such as dwell time and flight time [2, 3]. Third, we use a more controlled environment tailored to three specific types of cheating. Each of the approximately 20 participants will record one 10-minute baseline session of normal gameplay, followed by three 5-minute adversarial sessions in which they use KillAura, Flight, and X-Ray in turn. This design allows us to compare each player’s own baseline behavior against their behavior while cheating, directly isolating the irregularities that these methods introduce into a player’s biometric data. By focusing on the three most prevalent [1] methods of cheating in Minecraft: Java Edition, we can target our feature engineering

and model design more precisely rather than attempting to address a wide and continuously growing variety of cheating methods.

4 Primary Data Set

4.1 Data Collection Method

Data capture will be implemented before the input ever reaches the game engine. By doing this, we can record the actual raw interactions from the user rather than the filtered versions the game logic produces, which keeps our dataset as unfiltered as possible for the models to analyze.

4.1.1 Mouse Dynamics (External Python Logger)

Standard Minecraft gameplay is often limited by the server’s 20 tick per second update rate [2]. By using an external logger to track cursor position, it becomes possible to record mouse movement data at much higher polling rates depending on the participant’s hardware [2]. This data is critical for computing movement features such as velocity, acceleration, and jerk. While human hand movements naturally contain small variations and jitter along curved paths, automated systems like KillAura produce extremely straight or mathematically perfect trajectories that are difficult for a human to maintain [2].

4.1.2 Keystroke Dynamics (External Python Logger)

Keyboard input is recorded by capturing the exact timestamps of every key press and release event using the external logger. From this raw data, we can calculate two primary timing features: Dwell Time, which measures how long a key is held down, and Flight Time, which measures the delay between consecutive key presses. We focus on these because automated movement scripts like those used for Flight or Speed usually execute inputs at perfectly uniform intervals. In contrast, a real player’s timing will naturally shift and fluctuate based on what is happening in the game, making these timing rhythms a reliable way to identify scripts.

4.1.3 Participant Count & Data Volume Rationale

Originally we planned for 10–20 participants, but in the process of data gathering we shortlisted 5 people, extracted their data. But ended up with only 3 out of the 5 that had usable data from only three users: Ben, Sam, and Nnamdi. Data from two other participants had to be completely removed because of serious capture errors that would have corrupted the models. Although with a smaller sample size, since we split the data into 5 second intervals we ended up with plenty of data points for training. This approach, although slightly deficient with substantial user data, still produces a working ml-model. We chose to analyze individual interaction events because of the sheer volume of data

each user produces; even a brief session yields ample material for training. For instance, just twenty minutes of mouse movement can result in around 150,000 coordinate pairs [2]. This high data density is what allows models to identify distinct behavioral patterns and achieve high accuracy levels without requiring a large number of participants. Working with a smaller group of participants makes it possible to supervise the gameplay sessions directly, which is vital for ensuring the dataset is labeled accurately. This level of supervision allows us to know exactly when a cheat module is toggled on or off, which is essential for this type of experimental work.

4.1.4 Experimental Design: Baseline vs. Adversarial Sessions

The study is set up as a within-subject design. Since each participant completes both the clean and cheated sessions, we can use their own normal behavior as a control [2]. This makes it much easier to observe exactly how their interaction patterns change when cheating begins.

4.1.5 Baseline Session

To establish a clean baseline, each participant’s session is recorded for 10 minutes of normal gameplay without any cheats. This produces a stable biometric signature that can then be used as a reference for their typical movement and timing during activities such as PvP combat or open-world exploration.

4.1.6 Adversarial Sessions

The adversarial phase is split into three 5-minute blocks to test KillAura, Flight, and X-Ray individually. By isolating each cheating method, the system can more accurately identify how different forms of automation alter a user’s input behavior when compared directly to their natural baseline signature.

5 Model Building and Evaluation

5.1 Preprocessing

Before feature engineering, the data was cleaned to remove inconsistencies and anomalies, including incorrect data types and idle periods, to prevent errors in the downstream machine learning models.

- **Bad Data:** We recorded five gameplay sessions (Test Run, Baseline, Flight cheat, X-Ray cheat, and KillAura cheat) from five users. Two of the five recordings had capture problems that rendered the data unusable. Only the usable data was retained, and the rest was discarded.
- **The 5-Second Window:** Each gameplay session was divided into fixed 5-second windows as follows:
 1. Test Run: 5 minutes (any specified cheat used is noted)

2. Baseline: 10 minutes (no cheats used; legitimate gameplay only)
3. Flight Cheat: 10 minutes (Flight cheat used only)
4. X-Ray Cheat: 10 minutes (X-Ray cheat used only)
5. KillAura Cheat: 10 minutes (KillAura cheat used only)

A single 10-minute session produces one mouse CSV file and one keyboard CSV file. These recordings were split into fixed 5-second windows, with each window treated as an independent sample. This allows the system to determine whether cheating occurred during that specific short period of play.

5.1.1 Idle/AFK Filtering

During typical user gameplay session, there might be periods of time where the user is not actively interacting/playing the game. These idle periods typically produce long gaps between input events which end up in the raw dataset. If these gaps were to be left in the dataset, there is a chance an ML model tries to associate the gaps in users sessions rather than the actual gameplay action is. A threshold between 2 and 5 seconds is generally appropriate for Minecraft sessions given its fast-paced gameplay where legitimate play times don't often produce large gaps between input events. [2] After experimenting with timings, we found for our project that a 7 second threshold worked best. After idle sections are removed, the remaining events will be re-indexed with continuous timestamps so that timing features such as inter-event intervals are not artificially inflated.

5.2 Feature Engineering

Since we were dealing with two separate datasets (mouse and keyboard) and due to the idea being to train/test each model separately and then average the results, we extracted two separate feature sets strictly based on the raw data from the mouse logger (timestamp, event_type, and event_info) and the keyboard logger (timestamp, key_name, event_code, and scan_code). All features were computed over fixed 5-second windows.

Keyboard Feature Engineering:

- `mean_dwell` (Average dwell time in window)
- `var_dwell` (Variance of dwell time in window)
- `dwell_uniformity` (Standard deviation / Mean of dwell time)
- `mean_flight` (Average flight time in window)
- `var_flight` (Variance of flight time in window)
- `flight_uniformity` (Standard deviation / Mean of flight time)
- Event density (Count of key events that occurred in the window)

Mouse Feature Engineering:

- `path_length` (Total Euclidean distance covered)
- `straightness_ratio` (Ratio of straight line distance to total path length)
- `mean_speed` (Average speed of mouse movement)
- `max_jerk` (maximum jerk / change in acceleration)
- `mean_angle_change` (Average change in movement angle)
- `std_angle_change` (Standard deviation of angle changes)
- `heading_stability` (Variance of raw movement angles)

5.2.1 KillAura: Path Curvature and Aiming Patterns

The use of KillAura cheats involves the client/cheat automating targeting and attacking nearby entities, generally through the use of jarringly fast and precise movements that humans typically cannot replicate. Human aiming behavior generally involves a slightly curved or jittery mouse paths with very minute corrections as the player tracks the target. [2, 4]. These differences are able to be detected by computing some path-based features from the mouse trajectory over a window of consecutive events. From each window we will be able to determine path length, straight line distance, straightness ratio, and angular change. Human controlled aiming generally involves moderate curving and abnormal angular changes as the player typically tracks and overshoots a given target. We expect KillAura to produce windows with consistently high straightness ratios and repeated angular changes reflecting the more precise/mechanical behavior of an automated targeting system.

5.2.2 Flight: Dwell Time and Flight Time Rhythms

Flight cheats are a popular method of cheating in Minecraft, enabling players to move through the air with machine like input patterns. In normal gameplay, movement keys such as W, A, S, D, Space, and Shift are pressed and released with natural variation in timing, dependent on the in game situation. Dwell time and flight time are two keystroke dynamics that are useful for characterizing typing and movement rhythms. For movement related keys, we will calculate the dwell time and flight time for each key event pair across the session, then derive summary statistics such as the mean and variance. In normal gameplay, these inputs show natural variability, with players pressing at slightly different intervals and holding movement keys for varying durations depending on the environment, such as navigating hills or water. Flight cheats, however, tend to produce larger variation in jump intervals and more uniform press frequencies. By focusing on these timing features, the model should be able to detect overly consistent keystroke patterns that indicate automated movement, while treating greater variability as characteristic of natural, human controlled input.

5.2.3 X-Ray: Path Directness and Mining Behaviour

X-Ray allows users to see hidden blocks and structures nearby, such as ores, mineshafts, and lava, that would otherwise be obscured by surrounding blocks such as dirt and cobblestone. A player using X-Ray typically moves directly and efficiently toward these structures or valuable blocks, bypassing undesirable resources. A normal player, by contrast, explores more broadly, exhibits more directional variation, and mines a mix of valuable and common blocks. To capture this behavior, we will analyze the player’s movement path over several windows. For each window, we will examine path directness, heading stability, and action density. By analyzing these features, the model should learn to recognize sessions in which the player follows unusually direct and efficient paths toward high value targets, which are key indicators of X-Ray use, as opposed to the more exploratory and varied movement patterns of legitimate gameplay.

5.3 Algorithm Selection

At the start of the project we planned to use Random Forest and XGBoost algorithms primarily due to familiarity and promising results from other people online. Random Forest and SVM yielded null results and were not able to flag anything. XGBoost was deemed too complex for our local setup. Due to these limitations, we made the decision to switch entirely to LightGBM for both the keyboard and mouse classifiers. Typically LightGBM has faster training times and higher accuracy rates than RandomForest, XGBoost, and SVM. We trained two LightGBM classifiers on mouse and keyboard data respectively. We then took the probability/outputs and averaged them together to get our resulting prediction.

5.4 Testing Process

The data gathered from each test session was divided into 5-second windows. The keyboard and mouse models each produced a suspicion probability score for every window. These scores were averaged, and a window was flagged as suspicious if the maximum averaged probability met or exceeded the 0.5 threshold (otherwise it defaulted to baseline). If more than 50% of the windows in the full session were flagged as suspicious, the entire session was labeled as cheating. The primary cheat type was determined from the most common predicted label among the suspicious windows.

6 Success Metrics

Although overall accuracy is a useful starting point, it is not the only metric we plan to use to evaluate the system.

6.1 The Critical Imbalance of Error

Due to the fact that we are evaluating sessions which were not able to be perfectly controlled, and that the hopes would be to use this system to evaluate live gameplay and successfully act on the results. We need to recognize the real world impact of incorrect predictions. The goal is to detect as many cheaters as possible, but the cost of an incorrect prediction can vary. It could mean loss of users for a server, potentially loss of time for the user, and much more. These consequences therefore are a primary factor in how we fine-tune the models abilities.

6.1.1 False Negative

A false negative is when a system fails to detect a case where it should have. In our case this looks like a system identifying a cheating session as a baseline session. This weakens the accuracy of the model and may frustrate other players in game as they may identify it as the rules are not being enforced. A false negative reflects the limitations of a system and reduces the overall reliability. Nonetheless a false negative is deemed as less serious for our case in comparison to a false positive.

6.1.2 False Positive

A false positive is when a system incorrectly identifies a "good" case. In the context of our system, it would be our system flagging a baseline session as a cheating session. This is much more critical and crucial to mitigate/avoid as the consequences discussed in section 6.1 highlight the severity of the event of a false positive.

7 Results

It is also important to note that the output we received was based on the feature engineered/preprocessed data obtained from three users/players that were established prior in the research. Below is a representation of the Keyboard and Mouse cross-validation accuracy. It highlights the keyboard models higher accuracy in comparison to the mouse model which was considerably lower and had much larger variation:

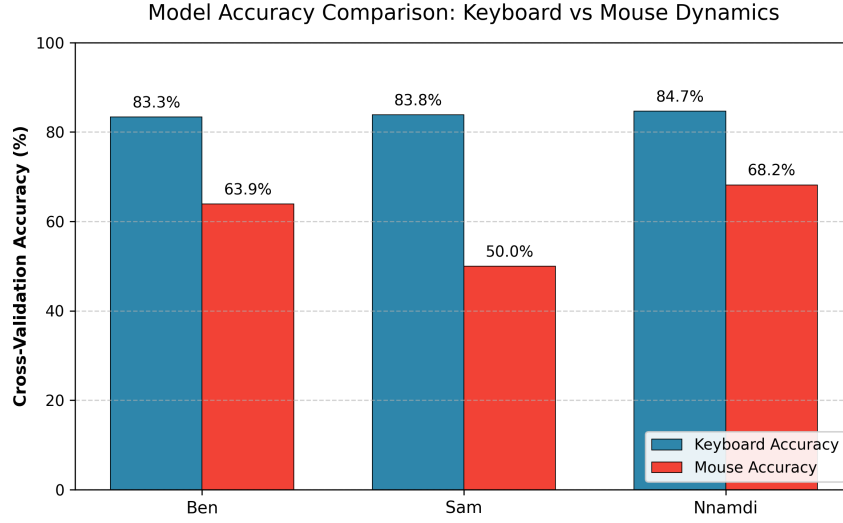


Figure 1: Model Accuracy Comparison: Keyboard vs Mouse Dynamics

We implemented two thresholds for our project. The first was a window-by-window threshold, where if a window was deemed more than 50% suspicious, then the window would be flagged as "cheating". The second threshold is a session-by-session based threshold where if 50% of the windows in a session are labelled as cheating, the entire session is categorized as cheating. The results varied as shown in the table and bar chart below:

Name	Session Type	Suspicion Percentage	Output Details
Ben	Baseline	40.82%	Correctly labeled legitimate
Ben	Flight	77.97%	Cheat detected (primary label: KillAura)
Ben	KillAura	72.41%	Cheat detected (primary label: Flight)
Ben	X-Ray	73.17%	Cheat detected (misclassified as Flight)
Sam	KillAura	75.00%	Cheat detected (primary label: Flight)
Nnamdi	Flight	38.46%	Incorrectly labeled legitimate (false negative)

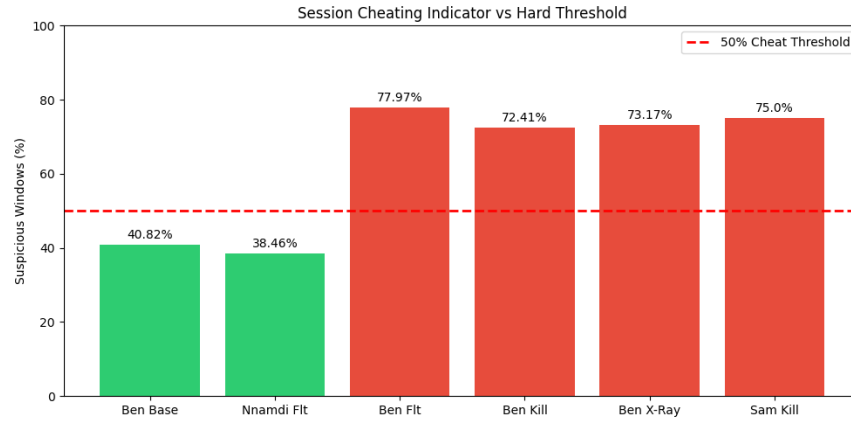


Figure 2: Session Cheating vs Threshold

The bar chart clearly shows the 50% threshold line. Any session above the red dashed line is flagged as cheating, while sessions below the line are classified as legitimate. Most of the cheat sessions (Flight, KillAura, and even X-Ray) exceeded the 50% threshold, whereas the baseline sessions remained below it. The one clear false negative occurred in Nnamdi's Flight session, which reached only 38.46% suspicious windows. Ben's logs include Baseline, Flight, KillAura, and X-Ray sessions. Sam's logs covered only the KillAura session, and Nnamdi's covered only the Flight session. More importantly, our conclusions regarding X-Ray detection and full baseline behaviour rely primarily on Ben's complete test set.

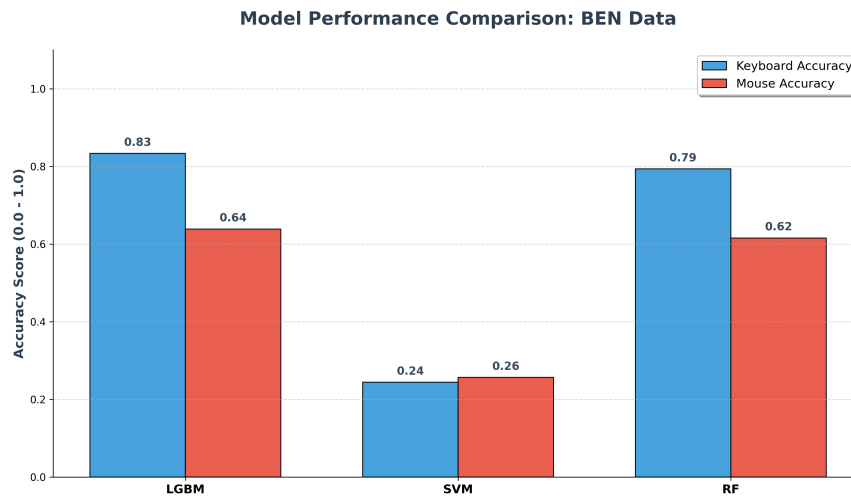


Figure 3: Comparison On Models Performance

The bar chart above highlights that keyboard dynamics had a better turnover on all three tested models excluding SVM where results were extremely close, though it had a higher accuracy on the mouse model. However, overall the LightGBM model achieved the highest accuracy at 0.83, followed by Random Forest at 0.79. Both of these models showed lower accuracies for mouse dynamics, recorded at 0.64 and 0.62 respectively. These findings indicate that keyboard features offer more reliability and much more distinction in the behaviors of a user between baselines and cheating sessions.

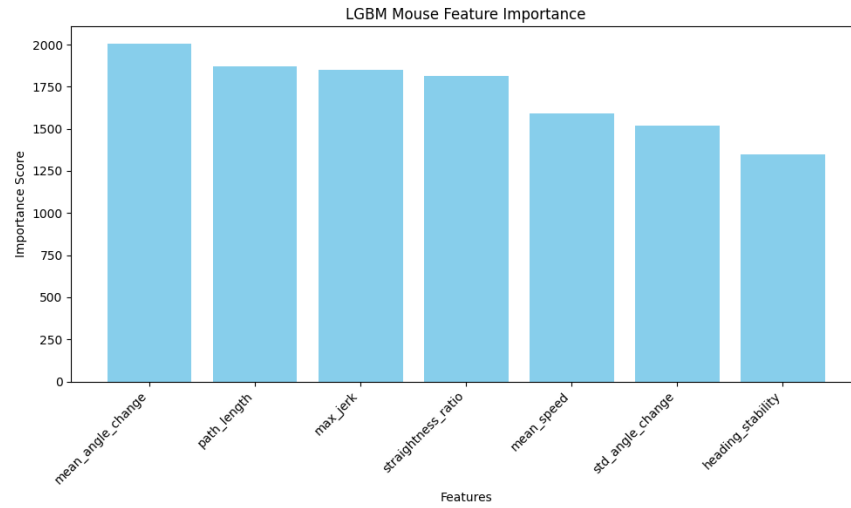


Figure 4: LGBM Mouse Feature

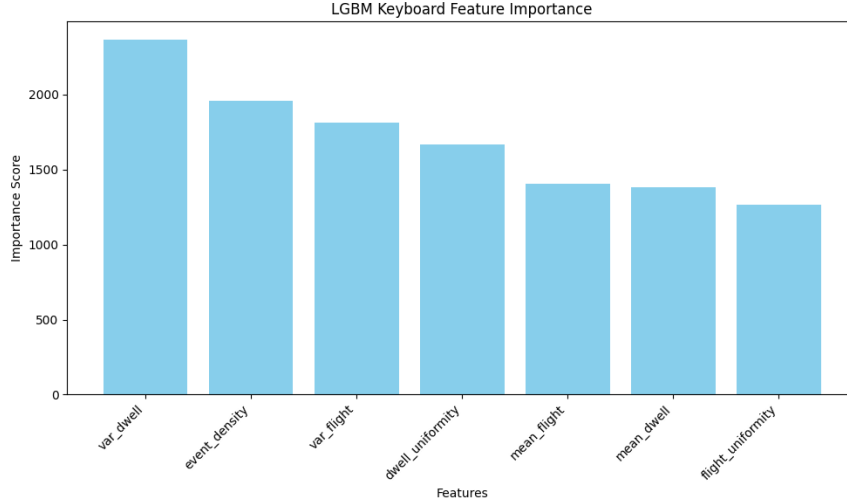


Figure 5: LGBM Keyboard Feature

The bar charts above, based on Ben’s data show how important the extracted features were in comparison to each other. For keyboard dynamics, dwell-time variance (var dwell) is the most important predictor, followed by event density and flight-time variance. These metrics show the irregularities in the timing rhythms that set human keystroke behavior apart from the unnaturally uniform inputs generated by automated cheats. In the mouse pipeline, the mean angle change is the most important factor. Path length, maximum jerk, and straightness ratio all support this. This shows the jagged and repetitive paths that are common in KillAura and Flight hacks. The keyboard features work better and more consistently than the mouse features, which is why the keyboard accuracy (0.83) was much higher than the mouse accuracy (0.64) in the BEN performance test.

8 Analysis of Result

The keyboard model produced better results than the mouse model across all three users. Mouse movement patterns, however, were heavily affected by background noise inherent in Minecraft gameplay. Because baseline play requires constant mouse movement, factors such as hand fatigue, variations in desk surface, and differences in mouse polling rates introduced significant variability. This noise made it substantially more difficult for the model to learn clean, discriminative patterns from the mouse features. KillAura and Flight cheats were detected more reliably because they impose robotic or highly repetitive input patterns that are difficult for a human player to replicate naturally. In contrast, X-Ray cheating proved to be the most challenging. This cheat does

not significantly alter mouse movement or keystroke dynamics; it primarily influences the direction and efficiency of the player’s mining path. Since the research focused exclusively on user behavioural biometrics rather than game-state or world knowledge, the model struggled to distinguish X-Ray gameplay from regular baseline play or even Flight sessions. The false negative observed in Nnamdi’s Flight session which fell below the 50threshold illustrates the issue with the fixed threshold. In this case, the cheating behaviour was sufficiently similar to the user’s normal playstyle that it was not flagged. This shows the need for player-specific threshold or more flexible decision rules in future iterations. Overall, the system successfully shows that behavioural biometrics can detect certain types of cheating in Minecraft.

9 Prediction and Future Work

The goal of the research we have conducted was to show that behavioral biometrics can be used to detect various methods of cheating in Minecraft. However, we were able to identify several limitations with the achieved results that indicate further work is needed to have a more reliable system. The current implementation has the same 50% thresholds for each user, but we would have been able to fine tune these thresholds to sway results for each users test session. This arises because the behavior of a video game is drastically different for each user, so not everyone’s data would work with the applied thresholds. There are varying areas for future development including a more generalized approach, threshold tweaking, feature tweaking, and much more. The following directions show particular promise for future work:

1. **Dataset Expansion:** The current study relied on a small dataset from only three usable participants. To improve accuracy and robustness, future work will collect gameplay data from 10–15 or more participants with varied hardware setups, playstyles, and levels of Minecraft experience. A larger and more diverse dataset will enable the models to learn more robust and generalizable patterns [5].
2. **Real-Time Tracking:** In future work, we intend to implement real-time detection, where the system continuously monitors raw mouse and keyboard data during live gameplay and flags suspicious activity immediately. This capability would support immediate intervention on multiplayer servers and better align with practical anti-cheat requirements. Note that the current implementation remains strictly offline and limited to post-session analysis and flagging.
3. **Generalized Detection Method:** We plan to develop a generalized model that learns common cheating signatures (e.g., KillAura or Flight) while gradually adapting to each player’s individual baseline behaviour over time. This approach would eliminate the need for manual per-user threshold tuning and help reduce both false positives and false negatives.

4. **Experimenting with Other Models:** We initially considered Random Forest and XGBoost, and also tested SVM, which yielded null results and was unable to flag cheating behaviour. We ultimately adopted LightGBM for its superior performance on our hardware. For future implementations, we plan to evaluate additional models to better capture subtle differences in player behaviour.

By pursuing these improvements, the system could achieve reliable detection of all tested cheats and evolve into a more robust behavioural biometric anti-cheat solution for Minecraft Java Edition and similar games.

10 Bibliography

References

- [1] Zolt, “[Guide] The Complete Guide to Hypixel Atlas!” June 2021. [Online]. Available: <https://hypixel.net/threads/guide-the-complete-guide-to-hypixel-atlas.4306633>
- [2] N. Siddiqui, R. Dave, and N. Seliya, “Continuous authentication using mouse movements, machine learning, and minecraft,” October 2021.
- [3] I. Da Silva Beserra, L. Camara, and M. Da Costa-Abreu, “Using keystroke and mouse dynamics for user identification in the online collaborative game league of legends,” in *7th International Conference on Imaging for Crime Detection and Prevention (ICDP 2016)*, 2016, pp. 1–6.
- [4] R. S. Kaminsky, M. Enev, and E. Andersen, “Identifying game players with mouse biometrics,” 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:5644670>
- [5] J. P. Pinto, A. Pimenta, and P. J. Novais, “Deep learning and multivariate time series for cheat detection in video games,” *Machine Learning*, vol. 110, no. 4, pp. 3037–3057. Accessed April 15, 2026, October 2021. [Online]. Available: <https://doi.org/10.1007/s10994-021-06055-x>